Bug Summary Reports

Rest API Todo List Manager (runTodoManagerRestAPI-1.5.5.jar), http://localhost:4567

Tester: Shahzoab Shafi (ID 261160765)

Exploratory session: Projects endpoints

Each bug uses the same form: an executive summary of 80 characters or less, a description, the potential impact on the operation of the system, and the steps to reproduce. Numbering continues from the Todos session (bugs 1 to 10). Bugs 11 to 19 were found in the Projects exploratory session; the DELETE variant of bug 15 was found while writing the unit test suite. Every bug maps to one failing "expected-behavior" test and one passing "actual-behavior" test in KnownBugsTest.

Overview

#	Executive summary	Severity	Failing unit test
11	PUT /projects/:id with only a title resets fields and drops task links	High	bug11_expected
12	POST /projects accepts a missing or empty title (201 Created)	Medium	bug12_expected
13	GET /projects/:id/categories returns 200 OK for a non-existent project	Medium	bug13_expected
14	Project-to-category link is not visible from /categories/:id/projects	High	bug14_expected
15	X-HTTP-Method-Override bypasses 405 on /projects endpoints (201)	High	bug15_expected
16	POST /projects with an array title silently keeps only the last element	Medium	bug16_expected
17	GET/POST /projects/:id/tasks/:id return 404 instead of 405	Low	bug17_expected
18	A POST /projects rejected for an invalid boolean still consumes an id	Low	bug18_expected
19	Malformed JSON/XML errors are sent with Content-Type text/html	Low	bug19_expected

Bug #11

Executive summary	**PUT /projects/:id with only a title resets fields and drops task links**
Pre-conditions	Server running with default data. Project 1 exists and is linked to todos 1 and 2.
Description	PUT /projects/:id is documented as amending a project. When the body only contains a title, the server answers 200 OK and renames the project, but it also resets description to "" and active/completed to "false", and removes every relationship between the project and its todos. The removed links also disappear from the todos' side (tasksof).
Potential impact	Silent data loss with a success code. A client that only wants to rename a project loses its description and all task links without any warning, and the todos lose their project. Same defect as bugs 7 and 8 on todos, so it affects every entity.
Steps to reproduce	**1.** curl -i -X POST http://localhost:4567/projects/1 -H "Content-Type: application/json" -d '{"description":"keep me"}' **2.** curl -i -X PUT http://localhost:4567/projects/1 -H "Content-Type: application/json" -d '{"title":"Renamed"}' **3.** curl -i http://localhost:4567/projects/1 and observe description "" and no tasks field. **4.** curl -i http://localhost:4567/todos/1/tasksof and observe an empty projects list.
Expected result	200 OK; title changed, description "keep me" and the links to todos 1 and 2 kept.
Actual result	200 OK; description reset to "", tasks relationship removed on both sides.
Severity	High
Frequency	Occurs every time.
Found by	Session notes (Projects), notes 21, 22 and 25
Automated test	KnownBugsTest.bug11_expected_putWithOnlyATitleKeepsTheOtherFields

Bug #12

Executive summary	**POST /projects accepts a missing or empty title (201 Created)**
Pre-conditions	Server running with default data.
Description	POST /projects with an empty body {} or with "title":"" returns 201 Created and creates a project whose title is empty. POST /todos rejects the same inputs with 400 ("title : can not be empty"), so projects apply no title validation at all.
Potential impact	Projects without a name can be created, which makes them impossible to identify in lists or filters. The validation rules are inconsistent between entities of the same API.
Steps to reproduce	**1.** curl -i -X POST http://localhost:4567/projects -H "Content-Type: application/json" -d '{}' **2.** curl -i -X POST http://localhost:4567/projects -H "Content-Type: application/json" -d '{"title":""}' **3.** Observe 201 Created twice, both with "title":"".
Expected result	400 Bad Request with a validation message; no project created.
Actual result	201 Created; a project with an empty title is created each time.
Severity	Medium
Frequency	Occurs every time.
Found by	Session notes (Projects), notes 7 and 8
Automated test	KnownBugsTest.bug12_expected_missingOrEmptyTitleIsRejected

Bug #13

Executive summary	**GET /projects/:id/categories returns 200 OK for a non-existent project**
Pre-conditions	Server running. No project with id 999999 exists.
Description	Requesting the categories of a project that does not exist returns 200 OK with an empty categories collection instead of 404 Not Found. GET /projects/999999 correctly returns 404, so the relationship endpoint is inconsistent with the instance endpoint.
Potential impact	Clients cannot tell a missing project from a project with no categories, so errors such as a wrong id go unnoticed. Same class of defect as bugs 3 and 9.
Steps to reproduce	**1.** curl -i http://localhost:4567/projects/999999 **2.** curl -i http://localhost:4567/projects/999999/categories **3.** Observe 404 for the project but 200 {"categories":[]} for its categories.
Expected result	404 Not Found with an error message.
Actual result	200 OK with {"categories":[]}.
Severity	Medium
Frequency	Occurs every time.
Found by	Session notes (Projects), note 37
Automated test	KnownBugsTest.bug13_expected_categoriesOfUnknownProjectReturns404

Bug #14

Executive summary	**Project-to-category link is not visible from /categories/:id/projects**
Pre-conditions	Server running. A project and a category exist.
Description	After linking a category to a project with POST /projects/:id/categories (201 Created), GET /projects/:id/categories lists the category, but GET /categories/:id/projects does not list the project. The relationship is stored on one side only, unlike project/todo links, which are visible from both sides.
Potential impact	The two sides of the data disagree. A client that navigates from a category will never find the projects linked to it, and deleting or reporting by category gives wrong results.
Steps to reproduce	**1.** curl -i -X POST http://localhost:4567/projects/1/categories -H "Content-Type: application/json" -d '{"id":"1"}' **2.** curl -i http://localhost:4567/projects/1/categories and observe category 1. **3.** curl -i http://localhost:4567/categories/1/projects and observe {"projects":[]}.
Expected result	GET /categories/1/projects lists project 1.
Actual result	GET /categories/1/projects returns an empty list.
Severity	High
Frequency	Occurs every time.
Found by	Session notes (Projects), notes 38 and 39
Automated test	KnownBugsTest.bug14_expected_linkedProjectIsListedUnderTheCategory

Bug #15

Executive summary	**X-HTTP-Method-Override bypasses 405 on /projects endpoints (201)**
Pre-conditions	Server running. A project and a todo exist.
Description	PUT /projects/:id/tasks and DELETE /projects return 405 Method Not Allowed. When the same requests carry the header X-HTTP-Method-Override: POST, the server treats them as POST: the PUT creates the project/todo relationship and the DELETE creates a new project, both with 201 Created. The DELETE case was found while writing the unit tests.
Potential impact	Method restrictions can be bypassed, which may defeat proxies, firewalls or authorization rules that filter on the HTTP method. A DELETE request that creates data is especially surprising. Same defect as bug 6 on todos.
Steps to reproduce	**1.** curl -i -X PUT http://localhost:4567/projects/1/tasks and observe 405. **2.** curl -i -X PUT http://localhost:4567/projects/1/tasks -H "X-HTTP-Method-Override: POST" -H "Content-Type: application/json" -d '{"id":"2"}' **3.** curl -i -X DELETE http://localhost:4567/projects -H "X-HTTP-Method-Override: POST" -H "Content-Type: application/json" -d '{"title":"ovr"}' **4.** Observe 201 Created for both, a new link and a new project.
Expected result	405 Method Not Allowed; nothing created.
Actual result	201 Created; the relationship and a new project are created.
Severity	High
Frequency	Occurs every time.
Found by	Session notes (Projects), note 42; DELETE variant found by unit tests
Automated test	KnownBugsTest.bug15_expected_methodOverrideIsIgnoredOnProjects

Bug #16

Executive summary	**POST /projects with an array title silently keeps only the last element**
Pre-conditions	Server running with default data.
Description	When POST /projects receives a title that is a JSON array (["a","b"]), the server answers 201 Created and stores only the last element ("b") as the title. The other elements are dropped without any warning.
Potential impact	Silent data loss and missing type validation. The client believes the request succeeded while the stored data differs from what was sent. Same defect as bug 1 on todos.
Steps to reproduce	**1.** curl -i -X POST http://localhost:4567/projects -H "Content-Type: application/json" -d '{"title":["a","b"]}' **2.** Observe 201 Created with "title":"b".
Expected result	400 Bad Request; no project created.
Actual result	201 Created; project created with title "b".
Severity	Medium
Frequency	Occurs every time.
Found by	Session notes (Projects), note 43
Automated test	KnownBugsTest.bug16_expected_arrayTitleOnProjectIsRejected

Bug #17

Executive summary	**GET/POST /projects/:id/tasks/:id return 404 instead of 405**
Pre-conditions	Server running. Project 1 is linked to todo 2.
Description	/projects/:id/tasks/:id only supports DELETE (OPTIONS answers Allow: OPTIONS, DELETE). GET and POST on an existing relationship return 404 Not Found with an empty text/html body, while PUT on the same URL correctly returns 405.
Potential impact	A 404 tells the client the relationship does not exist, which is false and misleading. Error handling is inconsistent across methods. Same defect as bug 5 on todos.
Steps to reproduce	**1.** curl -i http://localhost:4567/projects/1/tasks/2 **2.** curl -i -X POST http://localhost:4567/projects/1/tasks/2 **3.** curl -i -X PUT http://localhost:4567/projects/1/tasks/2 **4.** Observe 404, 404 and 405.
Expected result	405 Method Not Allowed for GET and POST.
Actual result	404 Not Found with an empty body.
Severity	Low
Frequency	Occurs every time.
Found by	Session notes (Projects), note 35
Automated test	KnownBugsTest.bug17_expected_getAndPostOnTaskLinkReturn405

Bug #18

Executive summary	**A POST /projects rejected for an invalid boolean still consumes an id**
Pre-conditions	Server running with default data.
Description	When POST /projects is rejected with 400 because active is not a boolean, no project is created, but the id counter is still incremented. The next valid project skips one id. Other rejected requests (malformed JSON, client-supplied id) do not consume an id.
Potential impact	Gaps in ids can confuse clients that expect consecutive ids and show that a failed validation leaves the server in a partially changed state.
Steps to reproduce	**1.** curl -i -X POST http://localhost:4567/projects -H "Content-Type: application/json" -d '{"title":"a"}' and note the id (N). **2.** curl -i -X POST http://localhost:4567/projects -H "Content-Type: application/json" -d '{"title":"bad","active":"yes"}' and observe 400. **3.** curl -i -X POST http://localhost:4567/projects -H "Content-Type: application/json" -d '{"title":"b"}' and observe id N+2.
Expected result	The next project receives id N+1.
Actual result	The next project receives id N+2.
Severity	Low
Frequency	Occurs every time.
Found by	Session notes (Projects), notes 10 and 12
Automated test	KnownBugsTest.bug18_expected_rejectedPostDoesNotConsumeAnId

Bug #19

Executive summary	**Malformed JSON/XML errors are sent with Content-Type text/html**
Pre-conditions	Server running.
Description	POST /projects with a malformed JSON or XML body correctly returns 400, but the Content-Type header of the response is text/html;charset=utf-8 although the body is JSON ({"errorMessages":[...]}). Other errors, such as 404 on GET /projects/999999, are correctly sent as application/json.
Potential impact	Clients that parse responses according to the Content-Type header cannot read the error message, and the API is inconsistent between error types.
Steps to reproduce	**1.** curl -i -X POST http://localhost:4567/projects -H "Content-Type: application/json" -d '{"title":"x"' **2.** curl -i -X POST http://localhost:4567/projects -H "Content-Type: application/xml" -d '<project><title>x' **3.** Observe Content-Type: text/html with a JSON body in both responses.
Expected result	400 with Content-Type: application/json.
Actual result	400 with Content-Type: text/html;charset=utf-8.
Severity	Low
Frequency	Occurs every time.
Found by	Session notes (Projects), notes 11 and 13
Automated test	KnownBugsTest.bug19_expected_parseErrorsAreSentAsJson